Week 07 – S2 PRACTICE: Sequelize Basics, Models & Relationships

Goal: Warm up with Sequelize fundamentals — connecting to a database, defining models, performing CRUD operations, and declaring 1–1 and 1–* relationships — before taking on the core exercises.

At the end of this practice, you should be able to…

- Use Sequelize Basics to connect to a MySQL database from Node.js.
- Define models and sync them with the database.
- Perform full CRUD operations using Sequelize methods.
- Declare and use 1–1 (One-to-One) and 1–* (One-to-Many) relationships.

How to start:

1. Download the start code from the MS Teams assignment.
2. Run `npm install` on both front and back projects.
3. Run `npm run dev` on both front and back projects.
4. Fill in the blank code blocks (`_____`) with the correct code.
5. Submit your code on the MS Teams assignment.

Resources:

- Official Sequelize docs — *https://sequelize.org/docs/v6*
- Sequelize with Node.js & MySQL — DigitalOcean tutorial
- Video: Sequelize crash course — YouTube
- Video: Sequelize associations — YouTube
- Video: Sequelize relationships — YouTube

Project Structure

Use the following modular folder structure for all exercises. Each layer has a single responsibility — Sequelize models live in `models/`, business logic in `controllers/`, route definitions in `routes/`, and the database connection in `config/`.

```
project/
│
├── config/
│   └── database.js          ← Sequelize instance + MySQL connection (Ex 1)
│
├── models/
│   ├── index.js             ← centralise all associations here
│   ├── User.js              ← 1–1 with Profile  (Ex 7)
│   ├── Profile.js
│   ├── Department.js        ← 1–* with Employee (Ex 6)
│   ├── Employee.js
│   ├── Student.js           ← *–* with Course   (Ex 8)
│   ├── Course.js
│   ├── Enrolment.js         ← junction table for Student ↔ Course
│   ├── Author.js            ← 1–* with Book     (Ex 10 core)
│   ├── Book.js
│   ├── Class.js             ← 1–* with AttendanceRecord (Ex 11 core)
│   └── AttendanceRecord.js
│
├── controllers/
│   ├── userController.js
│   ├── authorController.js
│   └── attendanceController.js
│
├── routes/
│   ├── userRoutes.js
│   ├── authorRoutes.js
│   └── attendanceRoutes.js
│
├── middleware/
│   └── logger.js            ← request logger (same pattern as Week 04)
│
├── .env                     ← DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, PORT
├── server.js                ← Express setup, mount routes, call sequelize.sync()
├── package.json             ← "type": "module"
└── README.md
```

Part A — Warm-ups · Sequelize Basics & CRUD

Exercise 1 – Install Sequelize & Connect to MySQL Warm-up

Goal: Scaffold a Node.js project, install Sequelize and the MySQL2 dialect, and verify that the connection succeeds.

Tasks:

Create the project and initialise it:

```
mkdir warmup-sequelize
cd warmup-sequelize
npm init -y
```

Install Sequelize and the MySQL2 driver. Fill in the blanks:

```
npm install ______ mysql2
```

Enable ES modules — add this to `package.json`:

```
{
  "type": "______"
}
```

Create `database.js` and fill in the blanks:

```
import { ______ } from 'sequelize';   // named import from 'sequelize'

const sequelize = new Sequelize('your_db', 'root', 'your_password', {
  host: '______',                     // usually 'localhost'
  dialect: '______',                  // 'mysql'
  logging: false,                     // silence SQL logs in the console
});

export default ______;
```

Create `index.js` and fill in the blank to test the connection:

```
import sequelize from './database.js';

try {
  await sequelize.______();           // authenticate() – tests the connection
  console.log('Database connected successfully!');
} catch (err) {
  console.error('Connection failed:', err);
}
```

Run it: `node index.js`.
Test: You should see "Database connected successfully!" in the terminal. Any error means the host, username, password, or database name is wrong.

Exercise 2 – Define Your First Model Warm-up

Goal: Learn how to map a database table to a JavaScript class using `sequelize.define()`.

Tasks:

Create `models/Product.js` and fill in the blanks:

```
import { DataTypes } from 'sequelize';
import sequelize from '../database.js';

const Product = sequelize.______(             // .define() maps a JS object to a tab
  '______',                                    // model name (Sequelize uses this as
  {
    name: {
      type: DataTypes.______,                  // STRING
      allowNull: false,
    },
    price: {
      type: DataTypes.______,                  // FLOAT
      defaultValue: 0,
    },
    stock: {
      type: DataTypes.INTEGER,
      defaultValue: ______,                    // 0
    },
  }
);

export ______ Product;
```

Import the model in `index.js` and log it to confirm the definition:

```
import Product from './models/Product.js';
console.log(Product.tableName);              // should print "Products"
```

Reflect: What is the difference between `DataTypes.STRING` and `DataTypes.TEXT`? When would you use each?

Exercise 3 – sync() the Table & create() Rows Warm-up

Goal: Understand how Sequelize creates (or updates) the physical table and how to insert rows with `Model.create()`.

In `index.js`, sync the model and fill in the blanks:

```
import sequelize from './database.js';
import Product from './models/Product.js';

// sync() compares the model definition with the real table.
// { force: true } drops and re-creates the table on every run (dev only!).
await sequelize._____({ force: true });
console.log('Tables synced!');
```

Insert two products using `create()`. Fill in the blanks:

```
const p1 = await Product._____({
  name: 'Laptop',
  price: 1299.99,
  stock: 10,
});

const p2 = await Product.create({ name: _____, price: 49.99, stock: 50 });
console.log('Created:', p1._____, p2.name);   // .name — access the model field
```

Test: Run `node index.js`. Open MySQL and run `SELECT * FROM Products;` — you should see the two rows.
Warning: `{ force: true }` drops the table every run. Use `{ alter: true }` or no option at all in production.

Exercise 4 – findAll() & findByPk() Warm-up

Goal: Read data from the table using the two most common Sequelize finders.

Add a function to `index.js` that lists all products. Fill in the blanks:

```
// Get all rows
const allProducts = await Product._____();
console.log('All products:', allProducts.map(p => p.name));
```

Find a single product by its primary key:

```
// findByPk(id) – look up the row whose PRIMARY KEY = id
const found = await Product._____(1);     // id = 1
if (found) {
  console.log('Found:', found._____);     // .name
} else {
  console.log('Not found');
}
```

Filter products using `findAll` with a where clause:

```
import { Op } from 'sequelize';

// Products cheaper than $100
const cheap = await Product.findAll({
  where: {
    price: { [Op._____]: 100 },   // Op.lt = less than
  },
});
console.log('Cheap products:', cheap.length);
```

Test: Each `console.log` should print the expected data from your seeded rows.

Exercise 5 – update() & destroy() Warm-up

Goal: Modify and remove rows using instance methods and static methods.

Update using an instance method — fill in the blanks:

```
// Option A: fetch the instance first, then save changes
const product = await Product.findByPk(1);
product._____ = 999.99;             // change the price field
await product._____();              // .save() persists the change
console.log('Updated price:', product.price);
```

Update using a static method (no need to fetch first):

```
// Option B: Product.update(values, { where })
const [affectedRows] = await Product._____(
  { stock: 0 },                     // set stock = 0
  { where: { price: { [Op.gt]: 500 } } }   // for products over $500
);
console.log('Rows updated:', _____);
```

Delete a row:

```
// Instance destroy
const toDelete = await Product.findByPk(2);
await toDelete._____();             // .destroy()
console.log('Deleted product with id 2');

// Static destroy (delete all out-of-stock)
await Product._____({ where: { stock: 0 } });
console.log('Cleared out-of-stock products');
```

Exercise 6 – One-to-Many Relationship (hasMany / belongsTo)

Context: A Department has many Employees. An Employee belongs to one Department.

Create `models/Department.js`:

```
import { DataTypes } from 'sequelize';
import sequelize from '../database.js';

const Department = sequelize.define('Department', {
  name: { type: DataTypes._____, allowNull: false },   // STRING
});
export default Department;
```

Create `models/Employee.js`:

```
import { DataTypes } from 'sequelize';
import sequelize from '../database.js';

const Employee = sequelize.define('Employee', {
  name:     { type: DataTypes.STRING, allowNull: false },
  jobTitle: { type: DataTypes.STRING },
});
export default Employee;
```

Declare the association in `index.js` and fill in the blanks:

```
import Department from './models/Department.js';
import Employee   from './models/Employee.js';

// One department has MANY employees
Department._____( Employee );        // .hasMany()
// Each employee BELONGS TO one department
Employee._____( Department );        // .belongsTo()

await sequelize.sync({ force: true });
```

Create sample data using the generated helper method:

```
const engineering = await Department.create({ name: 'Engineering' });

// .createEmployee() is generated by hasMany
await engineering._____({ name: 'Alice', jobTitle: 'Backend Dev' });
await engineering.createEmployee({ name: 'Bob',   jobTitle: 'Frontend Dev' });
```

Query all employees in a department:

```
const dept = await Department.findByPk(1, {
  include: [ ______ ],   // pass the Employee model to include its rows
});
console.log('Employees:', dept.Employees.map(e => e.name));
```

Exercise 7 – One-to-One Relationship (hasOne / belongsTo)

Context: Every User has exactly one Profile. A Profile belongs to one User.

Create models `User.js` and `Profile.js`:

```
// models/User.js
import { DataTypes } from 'sequelize';
import sequelize from '../database.js';

const User = sequelize.define('User', {
  username: { type: DataTypes.STRING, allowNull: false, unique: true },
  email:    { type: DataTypes.STRING, allowNull: false },
});
export default User;

// models/Profile.js
import { DataTypes } from 'sequelize';
import sequelize from '../database.js';

const Profile = sequelize.define('Profile', {
  bio:    { type: DataTypes.TEXT },
  avatar: { type: DataTypes.STRING },
});
export default Profile;
```

Declare the 1–1 association in `index.js`:

```
import User    from './models/User.js';
import Profile from './models/Profile.js';

// A User HAS ONE Profile
User._____( Profile );               // .hasOne()
// A Profile BELONGS TO a User
Profile._____( User );               // .belongsTo()

await sequelize.sync({ force: true });
```

Create a user and immediately attach a profile:

```
const user = await User.create({ username: 'sokha', email: 'sokha@example.com' });

// .createProfile() is generated by hasOne
await user._____({ bio: 'Backend developer', avatar: 'sokha.png' });
console.log('User created with profile.');
```

Read the user back with its profile:

```
const fullUser = await User.findByPk(1, {
  include: [ ______ ],   // include the Profile model
});
console.log(fullUser.username, '-', fullUser.Profile.bio);
```

Exercise 8 – Many-to-Many Relationship (belongsToMany)

Context: A Student can enrol in many Courses. A Course can have many Students. Junction table: `Enrolment`.

Create `Student.js`, `Course.js`, and `Enrolment.js`:

```
// models/Student.js (abbreviated)
const Student = sequelize.define('Student', { name: DataTypes.STRING });
// models/Course.js (abbreviated)
const Course = sequelize.define('Course', { title: DataTypes.STRING });
// models/Enrolment.js (junction table)
const Enrolment = sequelize.define('Enrolment', { grade: DataTypes.STRING });
```

Declare the Many-to-Many association:

```
// Student belongs to many Courses
Student.______( Course, { through: ______ });    // .belongsToMany(), Enrolment
// Course belongs to many Students
Course.______( Student, { through: Enrolment }); // .belongsToMany()
```

Enrol students in courses:

```
const alice = await Student.create({ name: 'Alice' });
const math  = await Course.create({ title: 'Mathematics' });

// .addCourse() is generated by belongsToMany
await alice.______( math, { through: { grade: 'A' } });   // addCourse()
```

Use `findAll` with include to load everything at once:

```
const allStudents = await Student.findAll({
  include: [{
    model: ______,           // Course model
    through: {
      attributes: ['______'] // only include the 'grade' column
    }
  }]
});
```

Part B — Core · Sequelize Relationships

Exercise 9 – Fix Broken Codes Core

Q1 – Broken Code 1

```
User.hasOne(Profile);
await sequelize.sync();
const profile = await Profile.create({ bio: 'Test' });
const user    = await profile.createUser({ username: 'joe' });
```

Problem	YOUR ANSWER
Fixed code	YOUR ANSWER

Q2 – Broken Code 2

```
Book.hasMany(Author);
await sequelize.sync();
const author = await Author.create({ name: 'Samnang' });
const book   = await author.createBook({ title: 'Wrong Way' });
```

Problem	YOUR ANSWER
Fixed code	YOUR ANSWER

Q3 – Broken Code 3

```
User.hasOne(Profile);
Profile.belongsTo(User);
const user    = await User.create({ username: 'Jon' });
const profile = await Profile.create({ bio: 'hello' });
await user.addProfile(profile);
```

Problem	YOUR ANSWER
Fixed code	YOUR ANSWER

Q4 – Broken Code 4

```
Employee.hasOne(Manager);
Manager.hasOne(Employee);
```

Problem	YOUR ANSWER
Fixed code	YOUR ANSWER

Exercise 10 – Authors & Books Core

Context: An author can write many books, but a book is written by one author.

Q1 – Define the Models and their Relationships

Model	Fields	Type
Author	name	string
	birthYear	integer
Book	title	string
	publicationYear	integer
	pages	integer

```
// Author has many Books
Author._____( Book );
// Book belongs to Author
Book._____( Author );
```

Q2 – Create Sample Data

```
const ronan = await Author.create({ name: 'Ronan The Best', birthYear: 1990 });
// Use .createBook() to attach books directly to an author
await ronan._____({ title: 'The Art of Code', publicationYear: 2018, pages: 320 });
await ronan.createBook({ title: 'Clean Architecture', publicationYear: 2021, pages:

// YOUR CODE HERE
```

Q3 – Queries

```
// Fetch all books by a given author
const author = await Author.findOne({ where: { name: 'Kim Ang' } });
const books  = await author._____();   // .getBooks()

// Create a new book for an existing author using .createBook()
const hokTim = await Author.findOne({ where: { name: 'Hok Tim' } });
await hokTim._____({ title: 'Young Genius', publicationYear: 2024, pages: 180 });

// List all authors along with their books (include)
const authorsWithBooks = await Author.findAll({
  _____: [Book],   // keyword that loads associated models
});
```

Exercise 11 – Attendance Tracker Core

Q1 & Q2 – Define Relationships

```
// A Student can have many attendance records
Student._____( AttendanceRecord );
AttendanceRecord._____( Student );

// A Class can have many attendance records
Class._____( AttendanceRecord );
AttendanceRecord._____( Class );

// A Student can attend many Classes through AttendanceRecord
Student._____( Class, { through: AttendanceRecord });
Class._____( Student, { through: AttendanceRecord });
```

Q3 – Write Code for the Following Operations

```
// Mark attendance for a student
await AttendanceRecord._____({
  date:      '2025-06-17',
  status:    'present',
  StudentId: 1,
  ClassId:   1,
});

// Get attendance for a student on a specific date
const record = await AttendanceRecord.findOne({
  where: { StudentId: 1, date: '_____' },   // fill in the date
  include: [ Class ],
});

// List attendance for all students in a class
const classRecords = await AttendanceRecord.findAll({
  where: { ClassId: 1 },
  include: [ _____ ],   // include Student model
});

// Get attendance summary for a student
const summary = await AttendanceRecord.findAll({
  where: { StudentId: 1 },
  include: [ Class ],
});
const presentCount = summary.filter(r => r.status === '_____').length;
```

Q4 – Develop a Functional REST API

Method	Route	Description
POST	/attendance?studentId=1&date=2025-06-17	Mark attendance
GET	/attendance?studentId=1&date=2025-06-17	Get specific attendance
GET	/classes/:id/attendance	List class attendance
GET	/students/:id/attendance	Get student summary

```
// POST /attendance?studentId=1&date=2025-06-17
app.post('/attendance', async (req, res) => {
  const { studentId, date } = req._____;   // read query params
  const { classId, status } = req.body;

  try {
    const record = await AttendanceRecord.create({
      date,
      status: status || 'present',
      StudentId: studentId,
      ClassId:   _____,              // classId
    });
    res.status(_____).json(record);  // 201 Created
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```